TIỂU LUẬN GIỮA KỲ

HỌC PHẦN: NHẬP MÔN HỌC MÁY

ĐỀ 5 — ENSEMBLE LEARNING

Bagging cây quyết định:
từ ví dụ tính tay đến thực nghiệm phân loại

Giảng viên: TS. Trần Lương Quốc Đại

Thành viên	Mã số sinh viên
Nguyễn Nghĩa Hiếu	52400013
Trần Quốc Bảo	52400102

Tháng 10 năm 2026

MỤC LỤC

MỞ ĐẦU ⋯ 3

1. NGUYÊN LÝ HOẠT ĐỘNG ⋯ 4

2. QUY TRÌNH VÀ VÍ DỤ TÍNH TAY ⋯ 6

3. BÀI TOÁN ỨNG DỤNG VÀ THIẾT KẾ THỰC NGHIỆM ⋯ 9

4. TRIỂN KHAI MÃ NGUỒN VÀ KẾT QUẢ ⋯ 11

KẾT LUẬN ⋯ 16

TÀI LIỆU THAM KHẢO ⋯ 17

Tóm tắt

Báo cáo tìm hiểu cách kết hợp nhiều mô hình để tạo một bộ phân loại. Trọng tâm là Bagging cây quyết định: lấy mẫu có hoàn lại, huấn luyện các cây và tổng hợp bằng đa số phiếu. Một ví dụ năm quan sát được tính xuyên suốt để thấy vì sao thay đổi mẫu huấn luyện có thể làm các cây đưa ra dự đoán khác nhau.

Phần thực nghiệm sử dụng dữ liệu Breast Cancer Wisconsin Diagnostic có sẵn trong scikit-learn. Bagging tự triển khai được đối chiếu với cây đơn và Random Forest trên 10 lần chia train/test có phân tầng. Accuracy trung bình của Bagging là 94,76 ± 1,59%, trong đó số sau dấu ± là độ lệch chuẩn mẫu tính theo điểm phần trăm. Kết quả cho thấy lợi ích trên dữ liệu khảo sát, đồng thời vẫn tồn tại lỗi và các giới hạn về đánh giá.

Từ khóa: Ensemble Learning; Bagging; bootstrap; hard voting; out-of-bag.

MỞ ĐẦU

Lý do chọn cách tiếp cận

Một cây quyết định dễ giải thích: mỗi nút đặt một câu hỏi về đặc trưng, quan sát đi theo nhánh phù hợp và nhận nhãn tại lá. Tuy nhiên, khi dữ liệu huấn luyện thay đổi, cấu trúc cây và dự đoán cũng có thể thay đổi đáng kể. Ensemble Learning khai thác nhiều mô hình thay vì phụ thuộc hoàn toàn vào một mô hình duy nhất [1, 2].

Bài chọn Bagging làm nội dung triển khai chính vì cơ chế của nó có thể quan sát trực tiếp: các mẫu bootstrap khác nhau tạo các cây khác nhau, sau đó đếm phiếu để ra quyết định. Cách làm này đủ đơn giản để tính tay nhưng vẫn thể hiện rõ bản chất của học tổ hợp.

Mục tiêu và phạm vi

- Giải thích vì sao cần nhiều mô hình, sự đa dạng giữa các mô hình và các cách tổng hợp dự đoán.
- Trình bày trọn vẹn một ví dụ: dữ liệu → bootstrap → cây → dự đoán → đa số phiếu, kèm cách hiểu out-of-bag.
- Tự triển khai bootstrap, hard voting và OOB; dùng cây quyết định của scikit-learn làm mô hình cơ sở.
- Đánh giá trên dữ liệu có sẵn, so sánh với cây đơn và Random Forest, giải thích kết quả và lỗi thay vì chỉ báo cáo accuracy.

Boosting, Voting và Stacking được giới thiệu để định vị Bagging trong Ensemble Learning. Báo cáo không nhận đã triển khai hoặc đo thực nghiệm Boosting/Stacking. Cây quyết định cơ sở và Random Forest sử dụng thư viện; phần lấy mẫu và tổ hợp của ManualBagging được viết tường minh trong mã nguồn.

Cách đọc báo cáo

Chương 1 trình bày nguyên lý. Chương 2 đi qua ví dụ tính tay. Chương 3 mô tả bài toán và cách đánh giá. Chương 4 giải thích mã nguồn, kết quả và nhận xét. Các công thức đều gắn với ý nghĩa hoặc con số cụ thể; phần thực nghiệm mang mục đích học thuật, không phải hệ thống chẩn đoán lâm sàng.

1. NGUYÊN LÝ HOẠT ĐỘNG

1.1. Ensemble Learning là gì?

Ensemble Learning kết hợp nhiều mô hình cơ sở thành một mô hình tổng hợp. Với phân loại, mỗi mô hình có thể đề xuất một nhãn hoặc một phân bố xác suất; bộ kết hợp dùng các đầu ra này để chọn nhãn cuối cùng. Ý tưởng không phải cứ có nhiều mô hình là tốt: lợi ích phụ thuộc cả chất lượng từng mô hình và việc chúng có mắc lỗi giống nhau hay không [2, 3].

Ví dụ, nếu ba mô hình cùng dự đoán sai ở đúng các quan sát giống nhau, bỏ phiếu không sửa được các lỗi đó. Nếu lỗi của chúng khác nhau và đa số thường đúng, tổ hợp có thể giảm tác động của một dự đoán sai riêng lẻ. Sự khác biệt hữu ích giữa các mô hình được gọi là tính đa dạng.

1.2. Bagging: tạo đa dạng bằng dữ liệu

Bagging là viết tắt của bootstrap aggregating. Cho tập train có N quan sát, ta lấy N lượt có hoàn lại để tạo một tập bootstrap. Có hoàn lại nghĩa là một quan sát vừa được lấy vẫn có thể được lấy tiếp: một số quan sát bị lặp, một số quan sát không xuất hiện trong tập mới. Lặp cách lấy mẫu này B lần và huấn luyện một mô hình trên mỗi tập [1].

Các cây trong Bagging được học độc lập về mặt thuật toán. Một cây không cần chờ kết quả hay lỗi của cây trước để học. Do cây quyết định nhạy với thay đổi dữ liệu, các tập bootstrap có thể tạo những phép chia khác nhau. Sau khi học xong, cùng một quan sát mới được đưa qua tất cả cây rồi tổng hợp dự đoán.

Trực giác giảm phương sai: nếu một thay đổi nhỏ của train làm một cây thay đổi mạnh, tổng hợp nhiều cây khác nhau có thể ổn định hơn. Bagging thường phù hợp với loại mô hình có tính biến động cao như cây quyết định. Báo cáo không suy từ trực giác này rằng mọi tổ hợp đều tốt hơn mô hình đơn.

1.3. Quy tắc hard voting

$$H(x) = \text{argmax theo lớp } c \text{ của tổng } I[h_b(x) = c],\ b = 1,\ldots,B$$

Trong đó h_b(x) là nhãn do cây b dự đoán; I bằng 1 nếu cây chọn lớp c và bằng 0 nếu không. Tổng các I chính là số phiếu của lớp c; argmax chọn lớp có nhiều phiếu nhất. Với hai phiếu ác tính và một phiếu lành tính, tổ hợp chọn ác tính. Khi hòa phiếu, mã chọn lớp đầu trong classes_; 51 cây tránh hòa ở bài toán hai lớp, nhưng số cây lẻ vẫn có thể hòa khi có nhiều hơn hai lớp.

1.4. Hard voting và soft voting

Hard voting chỉ đếm nhãn. Soft voting lấy trung bình xác suất dự đoán của các mô hình rồi chọn lớp có xác suất trung bình lớn nhất. Hai cách có thể cho kết quả khác nhau; soft voting cũng không tự động tốt hơn nếu các xác suất đầu vào không đáng tin cậy [3].

Mô hình	P(ác tính | x)	Nhãn riêng
1	0,51	Ác tính
2	0,51	Ác tính
3	0,01	Lành tính

Trong ví dụ giả định này, hard voting chọn ác tính vì có 2/3 phiếu. Soft voting tính (0,51 + 0,51 + 0,01)/3 ≈ 0,343 cho ác tính và 0,657 cho lành tính, nên chọn lành tính. Các số này minh họa quy tắc tổng hợp, không phải đầu ra của thực nghiệm Breast Cancer.

1.5. Những cách kết hợp thường gặp

Cách tiếp cận	Ý tưởng chính	Vai trò trong bài
Bagging	Học độc lập trên các bootstrap rồi tổng hợp	Trọng tâm tự triển khai
Boosting	Học tuần tự; mô hình sau xử lý sai sót hoặc giảm loss	Giới thiệu để đối chiếu
Voting	Kết hợp nhãn hoặc xác suất của nhiều mô hình	Hard voting của Bagging
Stacking	Dùng dự đoán ngoài fold để học bộ kết hợp	Giới thiệu để đối chiếu

Các cách trên không hoàn toàn tách rời: Bagging cũng cần quy tắc tổng hợp như voting. Với AdaBoost, một ví dụ của Boosting, các mẫu đang bị dự đoán sai nhận trọng số lớn hơn để mô hình sau chú ý hơn; trọng số được chuẩn hóa sau mỗi vòng [6]. Với Stacking, tầng kết hợp phải được học từ dự đoán ngoài fold của các mô hình cơ sở, tránh dùng dự đoán quá tốt ngay trên dữ liệu chúng đã học [7].

1.6. Random Forest và giới hạn của so sánh

Random Forest kết hợp bootstrap với việc chọn ngẫu nhiên một tập con đặc trưng tại từng phép chia, giúp tăng đa dạng giữa các cây [2]. Trong scikit-learn, RandomForestClassifier tổng hợp bằng trung bình xác suất lớp của các cây, còn ManualBagging trong bài tổng hợp bằng đa số nhãn. Do đó, đối chiếu hai mô hình không phải thí nghiệm chỉ thay đổi riêng một yếu tố chọn đặc trưng [3].

2. QUY TRÌNH VÀ VÍ DỤ TÍNH TAY

2.1. Quy trình Bagging

Bước	Thao tác	Điều cần giữ đúng
1	Chuẩn bị train và giữ riêng test	Test không tham gia học hay chọn cấu hình
2	Lấy B bootstrap, mỗi tập N lượt	Lấy có hoàn lại, không chia train thành B phần
3	Huấn luyện một cây trên mỗi bootstrap	Các cây học độc lập
4	Đưa cùng x qua mọi cây	Tất cả cây dự đoán cùng một quan sát
5	Đếm phiếu và chọn lớp đa số	Quy tắc hòa phiếu phải rõ ràng
6	Đánh giá test và đọc các loại lỗi	Không chỉ nhìn accuracy

2.2. Dữ liệu minh họa xuyên suốt

Dùng năm quan sát một chiều để có thể theo dõi từng con số. Đây là dữ liệu giả lập; nhãn ác/lành chỉ giúp thống nhất cách gọi, không mô tả đặc trưng y tế thực tế. Chỉ số hàng bắt đầu từ 0. Cây trong ví dụ chỉ được chia một lần (max_depth=1).

Chỉ số	0	1	2	3	4
Đặc trưng x	1	2	3	4	5
Nhãn	Lành (1)	Lành (1)	Ác (0)	Ác (0)	Ác (0)

Ta cố định ba bootstrap dưới đây để người đọc tái lập đúng phép tính. Trong thực nghiệm thật, chương trình sinh ngẫu nhiên các bootstrap theo seed; các chỉ số được cố định ở đây chỉ phục vụ giảng giải.

Cây	Chỉ số bootstrap	Hàng không có mặt
h1	[0, 0, 2, 4, 1]	[3]
h2	[0, 0, 1, 3, 4]	[2]
h3	[4, 2, 4, 1, 3]	[0]

2.3. Từ bootstrap đến phép chia của cây

Cây quyết định chọn phép chia làm các nhánh thuần nhất hơn. Với hai lớp, độ hỗn tạp Gini tại một nút được tính bằng 1 − p_ác² − p_lành². Nút chỉ có một lớp có Gini bằng 0. Gini sau chia là trung bình có trọng số theo số quan sát của hai nhánh [5].

$$\text{Gini sau chia} = (N_{trái}/N) \times \text{Gini}_{trái} + (N_{phải}/N) \times \text{Gini}_{phải}$$

Cây h1 dùng [0,0,2,4,1], tức các giá trị x là [1,1,3,5,2], với ba nhãn lành và hai nhãn ác. Gini trước chia bằng $1 - (3/5)^2 - (2/5)^2 = 0{,}48$. Nếu chọn ngưỡng $x \le 2{,}5$, nhánh trái có ba mẫu lành; nhánh phải có hai mẫu ác. Gini mỗi nhánh bằng 0, nên Gini sau chia bằng $(3/5)\times 0 + (2/5)\times 0 = 0$. Mức giảm bằng 0,48.

Vì sao ngưỡng là 2,5? Sau khi sắp các giá trị khác nhau thành 1, 2, 3, 5, các ngưỡng ứng viên nằm giữa hai giá trị liên tiếp. Ngưỡng 2,5 tách hoàn toàn hai lớp nên đạt mức giảm lớn nhất ở nút này. Các ngưỡng 1,5 và 4 không cho hai nhánh đều thuần nhất.

Cây	Nhãn trong bootstrap	Ngưỡng được học	Gini trước → sau
h1	3 lành; 2 ác	$x \le 2{,}5$	0,48 → 0,00
h2	3 lành; 2 ác	$x \le 3{,}0$	0,48 → 0,00
h3	1 lành; 4 ác	$x \le 2{,}5$	0,32 → 0,00

Đối với h2, bootstrap không có hàng x=3; hai giá trị sát ranh giới lớp là x=2 và x=4, nên cây chọn trung điểm 3,0. Cây h3 vẫn có x=2 và x=3, nên ngưỡng là 2,5. Cả ba cây đều phân loại đúng các mẫu trong bootstrap của riêng mình, nhưng ranh giới chúng học không hoàn toàn giống nhau. Đây là cơ chế tạo đa dạng mà ví dụ cần làm rõ.

Các ngưỡng và giá trị Gini trong bảng được kiểm tra bằng DecisionTreeClassifier(max_depth=1). Ví dụ không giả định trước phiếu của cây: phiếu ở bước tiếp theo được lấy từ chính ba cây vừa huấn luyện.

2.4. Dự đoán cùng một quan sát và cộng phiếu

Đưa cùng quan sát mới x=2,8 qua ba cây. Quy tắc mỗi cây là: x không vượt ngưỡng thì chọn lành; x lớn hơn ngưỡng thì chọn ác. Chưa biết nhãn thật của quan sát mới này, nên chỉ kết luận dự đoán, không tự gọi dự đoán là đúng.

Cây	So sánh	Dự đoán
h1	2,8 > 2,5	ác tính
h2	2,8 ≤ 3,0	lành tính
h3	2,8 > 2,5	ác tính

Số phiếu ác = 2; số phiếu lành = 1 → H(2,8) = ác tính (0)

Như vậy, dữ liệu và bootstrap đã dẫn tới ba ngưỡng thực, ba dự đoán thực rồi mới đến kết quả voting. Tỷ lệ 2/3 chỉ là tỷ lệ cây bỏ phiếu ác trong ví dụ; nó không phải xác suất mắc bệnh đã được hiệu chỉnh.

2.5. Out-of-bag: dùng mẫu cây chưa học

Một quan sát không có trong bootstrap của cây được gọi là out-of-bag (OOB) đối với cây đó. Khi đánh giá OOB cho một hàng train, chỉ những cây không học hàng ấy mới được bỏ phiếu. Không được dùng mọi cây để đánh giá chính dữ liệu train rồi gọi đó là OOB.

Hàng	Cây đủ điều kiện OOB	Thật → dự đoán	Kết quả
0 (x=1)	h3	Lành → Lành	Đúng
2 (x=3)	h2	Ác → Lành	Sai
3 (x=4)	h1	Ác → Ác	Đúng
1 và 4	Không có cây OOB	Không dự đoán	Loại khỏi accuracy OOB

Ở ví dụ nhỏ, coverage OOB = 3/5 = 60%; accuracy OOB trên ba hàng có phiếu = 2/3 ≈ 66,67%. Hàng 1 và 4 không được gán một lớp giả để tính điểm. Với N quan sát, xác suất một hàng không xuất hiện sau N lượt lấy mẫu là (1−1/N)^N; khi N lớn tiến gần 36,8%. Với N=5, giá trị chính xác là 32,768% [1].

3. BÀI TOÁN ỨNG DỤNG VÀ THIẾT KẾ THỰC NGHIỆM

3.1. Mục tiêu, dữ liệu đầu vào và đầu ra

Bài toán minh họa là phân loại mẫu khối u vú thành ác tính hoặc lành tính từ các đặc trưng đã trích xuất. Dữ liệu Breast Cancer Wisconsin Diagnostic được nạp bằng load_breast_cancer của scikit-learn [4]. Ta học ánh xạ từ một vector 30 giá trị số sang một trong hai nhãn.

Thuộc tính	Mô tả
Số quan sát	569
Đầu vào	30 đặc trưng số của nhân tế bào, như radius, texture, perimeter, area
Đầu ra	0 = malignant (ác tính); 1 = benign (lành tính)
Số lượng lớp	212 ác tính; 357 lành tính
Nguồn dùng trong mã	sklearn.datasets.load_breast_cancer; không cần tải dữ liệu riêng
Mục đích	Minh họa phương pháp học máy, chưa xác thực lâm sàng

Đầu vào không phải ảnh gốc và bài không huấn luyện mô hình nhận dạng ảnh. Các đặc trưng đã được chuẩn bị; cây sử dụng phép chia theo ngưỡng nên không yêu cầu chuẩn hóa thang đo như các mô hình dựa trên khoảng cách. Đây là lựa chọn phù hợp với một bài minh họa tập trung vào cơ chế tổ hợp.

3.2. Chia dữ liệu và giữ test độc lập với lựa chọn

Mỗi seed chia 75% train và 25% test, có stratify theo nhãn để giữ gần tỷ lệ hai lớp. Tại seed 42, train có 426 mẫu (159 ác và 267 lành), test có 143 mẫu (53 ác và 90 lành). Test không được dùng để chọn độ sâu, số cây hay seed tốt nhất.

Trên train, thử max_depth trong [2,3,4,5,6] bằng StratifiedKFold năm fold. Mỗi lần thử học trên bốn fold và chấm trên fold còn lại, rồi lấy trung bình balanced accuracy. Cấu hình có điểm cao nhất được chọn; nếu hòa, mã ưu tiên độ sâu nhỏ hơn. Sau đó huấn luyện mô hình trên toàn train và mới đánh giá test.

3.3. Cấu hình đối chiếu và lặp thí nghiệm

Mô hình	Cấu hình	Cách tổng hợp
Cây đơn	Một DecisionTreeClassifier	Dự đoán trực tiếp
Manual Bagging	51 cây; bootstrap tự viết	Đa số nhãn
Random Forest	51 cây; max_features='sqrt'	Trung bình xác suất cây

Ba mô hình dùng chung train/test và độ sâu vừa chọn bằng cây đơn để đối chiếu đơn giản. Đây chưa phải cấu hình tối ưu riêng cho từng ensemble. Với 30 đặc trưng, sqrt tương ứng mục tiêu 5 đặc trưng tại một phép chia của Random Forest; thư viện có thể xem thêm đặc trưng để tìm phép chia hợp lệ [3].

Thí nghiệm lặp cho các seed cố định 42–51. Mỗi seed chia lại dữ liệu và chọn lại độ sâu trên train của chính seed đó. Báo cáo giữ seed 42 làm lần tham chiếu để phân tích lỗi cụ thể; bảng tổng hợp sử dụng đủ cả mười seed, không chọn lượt có điểm đẹp nhất.

3.4. Chỉ số và quy ước lớp dương

Chọn ác tính = 0 làm lớp dương. Vì vậy, không được dùng mặc định pos_label=1 rồi gọi đó là recall ác tính. TP là mẫu ác được dự đoán ác; FN là mẫu ác bị dự đoán lành; FP là mẫu lành bị dự đoán ác; TN là mẫu lành được dự đoán lành. Ma trận có hàng là nhãn thật, cột là dự đoán, thứ tự [ác, lành].

Chỉ số	Công thức	Cách đọc
Accuracy	(TP+TN)/N	Tỷ lệ dự đoán đúng tổng thể
Precision ác	TP/(TP+FP)	Trong các dự đoán ác, bao nhiêu là ác thật
Recall ác	TP/(TP+FN)	Trong các mẫu ác thật, phát hiện được bao nhiêu
F1 ác	2TP/(2TP+FP+FN)	Cân bằng precision và recall
Balanced accuracy	(Recall ác + Recall lành)/2	Cân bằng đóng góp hai lớp; dùng chọn depth

Recall ác tính và số FN được chú ý vì chúng thể hiện các ca ác tính bị bỏ sót. Điều này không biến mô hình thành công cụ y tế đã được kiểm định. Khi tổng hợp nhiều seed, báo cáo mean và SD mẫu (ddof=1). Các tập test giữa các seed có thể giao nhau, nên SD chỉ mô tả dao động, không phải khoảng tin cậy độc lập.

4. TRIỂN KHAI MÃ NGUỒN VÀ KẾT QUẢ

4.1. Tổ chức mã và phần tự triển khai

Tệp / thành phần	Nhiệm vụ
ensemble_model.py / ManualBagging	Kiểm tra dữ liệu; lấy mẫu bootstrap; học cây; đếm phiếu; OOB
de_5_ensemble_learning.py	Chia train/test; chọn depth; so sánh ba mô hình; xuất kết quả và biểu đồ
examples/toy_bagging.py	Tái lập ví dụ năm quan sát và các phép tính trong Chương 2
tests/	Kiểm thử voting/OOB, tái lập, quy trình và xuất tệp

ManualBagging kế thừa giao diện estimator của scikit-learn để dùng được clone và cross-validation, nhưng không gọi BaggingClassifier để thực hiện thay việc lấy mẫu hay tổng hợp. Nhãn đầu vào được ánh xạ về các chỉ số liên tiếp bằng np.unique, giúp đếm phiếu đúng cả khi nhãn là chuỗi hoặc số không liên tiếp.

4.2. Lấy mẫu và huấn luyện

```
for _ in range(self.n_estimators):
    indices = rng.integers(0, n_samples, size=n_samples)
    tree = DecisionTreeClassifier(max_depth=self.max_depth,
                                  random_state=tree_seed)
    tree.fit(X[indices], encoded[indices])
    self.estimators_.append(tree)
```

Đây là trích lược để giải thích; tree_seed trong bản đầy đủ được sinh từ rng. rng.integers tạo đúng N chỉ số có hoàn lại. X và y phải lấy cùng indices để không lệch nhãn. Mỗi cây có seed riêng để tái lập; bản đầy đủ còn lưu các chỉ số bootstrap nhằm kiểm tra và tính OOB. Cây cơ sở do thư viện cung cấp, phần vòng lặp Bagging thuộc mã tự triển khai.

4.3. Đếm phiếu và chọn nhãn

```
counts = np.zeros((len(X), len(self.classes_)), dtype=np.int64)
rows = np.arange(len(X))
for tree in self.estimators_:
    predicted = tree.predict(X).astype(int)
    np.add.at(counts, (rows, predicted), 1)
return self.classes_[counts.argmax(axis=1)]
```

Mỗi hàng của counts ứng với một quan sát, mỗi cột ứng với một lớp. Mỗi cây tăng đúng một phiếu cho mỗi quan sát. argmax tìm cột có phiếu lớn nhất, sau đó classes_ ánh xạ về nhãn gốc. predict_vote_fraction chia các số phiếu cho số cây; tên hàm nhấn mạnh đây là tỷ lệ phiếu, không nhận là xác suất đã hiệu chỉnh.

4.4. OOB và các kiểm tra giúp kết quả đáng tin

Với mỗi bootstrap, np.bincount đếm số lần mỗi hàng xuất hiện. Các hàng có số đếm bằng 0 mới được cây dự đoán để tích lũy phiếu OOB. Khi cộng xong mọi cây, chỉ những hàng có ít nhất một phiếu mới tham gia accuracy OOB. Hàng chưa có phiếu giữ giá trị NaN trong oob_decision_function_; không coi chúng là dự đoán lớp 0.

Độ sâu đã được chọn bằng CV trên toàn train trước khi tính OOB. Vì thế OOB sau bước lựa chọn này không hoàn toàn độc lập với tuning; đây là chẩn đoán bổ sung. Kết quả test được giữ riêng vẫn là đánh giá cuối của mỗi seed.

Nhóm kiểm tra	Hành vi được kiểm tra
Dữ liệu và giao diện	Đầu vào sai; dự đoán trước fit; ánh xạ nhãn; clone và CV
Bootstrap và voting	Lấy có hoàn lại; mỗi cây đúng N lượt; tổng phiếu; xử lý hòa
OOB	Chỉ cây bỏ qua mẫu mới có phiếu; coverage; hàng chưa có phiếu
Quy trình thí nghiệm	Chọn depth chỉ nhận train; hồi quy kết quả seed 42
Xuất kết quả	CLI chạy thực; JSON chuẩn; CSV; ảnh PNG; số cây tùy chỉnh
Ví dụ tính tay	Ngưỡng/Gini thực; cùng x cho mọi cây; đúng nguồn phiếu OOB

4.5. Cách tái lập

Dùng Python 3.12 và cài requirements.txt. Các phiên bản trực tiếp của lần chạy được ghi trong results.json; việc ghim phiên bản cùng seed giúp hạn chế sai khác giữa môi trường. Dữ liệu đã nằm trong scikit-learn, không cần API key.

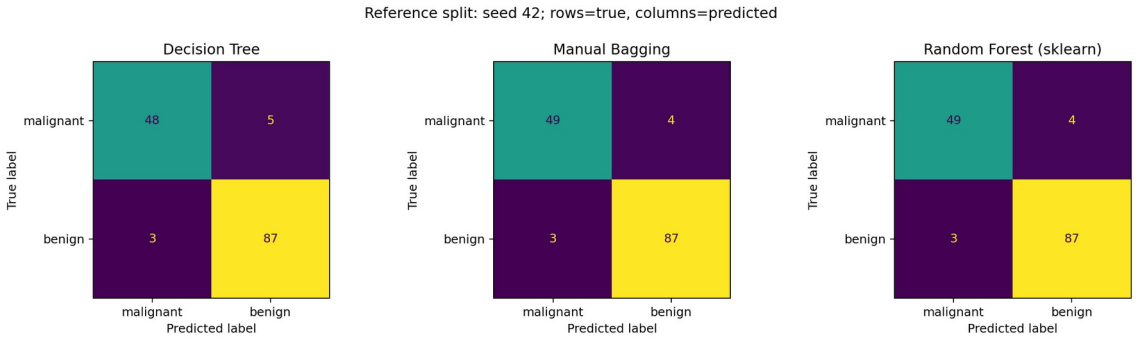
```
python3 -m venv .venv
.venv/bin/python -m pip install -r requirements.txt
.venv/bin/python -m unittest discover -s tests -v
.venv/bin/python examples/toy_bagging.py --output-dir outputs/toy
.venv/bin/python de_5_ensemble_learning.py
```

Trên Windows, dùng .venv\Scripts\python.exe thay cho .venv/bin/python. Trong cloud có home chỉ đọc, đặt MPLCONFIGDIR=/tmp/hocmaygk-matplotlib và XDG_CACHE_HOME=/tmp/hocmaygk-cache trước khi chạy. Demo nhanh dùng --repeats 1 --output-dir outputs/quick để không ghi đè kết quả đầy đủ. Bảng/biểu đồ trong báo cáo này được lấy từ cấu hình mặc định 10 seed, không suy ra từ demo một seed.

4.6. Kết quả lần tham chiếu: seed 42

Train có 426 mẫu, test có 143 mẫu. Độ sâu 3 được chọn vì balanced accuracy CV trung bình đạt 93,73%, cao nhất trong năm độ sâu ứng viên. Hai ensemble đều có 51 cây.

Mô hình	Accuracy	Precision ác	Recall ác	F1 ác
Cây quyết định đơn	94,41%	94,12%	90,57%	92,31%
Bagging tự triển khai	95,10%	94,23%	92,45%	93,33%
Random Forest (sklearn)	95,10%	94,23%	92,45%	93,33%



Hình 1. Ma trận nhầm lẫn của ba mô hình, seed 42; hàng thật, cột dự đoán.

Bagging dự đoán đúng 136/143 mẫu, cây đơn đúng 135/143. Chênh lệch là một mẫu, tương đương 0,70 điểm phần trăm accuracy. Không nên mô tả một chênh lệch nhỏ trên một lần chia như bằng chứng ưu thế tổng quát.

Thay số để kiểm tra chỉ số Bagging

Chỉ số	Thay số	Kết quả
Accuracy	(49+87)/143	95,10%
Precision ác	49/(49+3)	94,23%
Recall ác	49/(49+4)	92,45%
F1 ác	2×49/(2×49+3+4)	93,33%

Cây đơn bỏ sót 5/53 mẫu ác; Bagging còn bỏ sót 4/53 mẫu ác. Random Forest trùng các chỉ số tổng hợp của Bagging trong lượt này. Trùng số đếm không đủ để kết luận hai phương pháp tương đương hoặc dự đoán giống nhau trên mọi quan sát.

4.7. Độ ổn định trên mười lần chia

Bảng dưới lấy đủ các seed 42–51. Mỗi số trước dấu ± là trung bình theo %, số sau là SD mẫu theo điểm phần trăm. Vì các test có thể giao nhau, không diễn giải bảng thành khoảng tin cậy hay một kiểm định ý nghĩa thống kê độc lập.

Mô hình	Accuracy (%)	Recall ác (%)	F1 ác (%)
Cây quyết định đơn	92,80 ± 1,84	90,38 ± 4,66	90,30 ± 2,42
Bagging tự triển khai	94,76 ± 1,59	92,45 ± 3,77	92,89 ± 2,13
Random Forest (sklearn)	95,03 ± 1,85	92,26 ± 3,26	93,26 ± 2,39



Hình 2. Trung bình accuracy và recall qua 10 holdout; thanh sai số là SD mẫu.

Bagging có accuracy trung bình cao hơn cây đơn khoảng 1,96 điểm phần trăm và SD accuracy thấp hơn trong khảo sát này. Random Forest có accuracy trung bình cao hơn nhẹ so với Bagging, trong khi recall ác tính trung bình của Bagging cao hơn nhẹ. Vì vậy, nhận xét cần gắn với chỉ số đang xem, không chỉ tuyên bố một mô hình thắng mọi mặt.

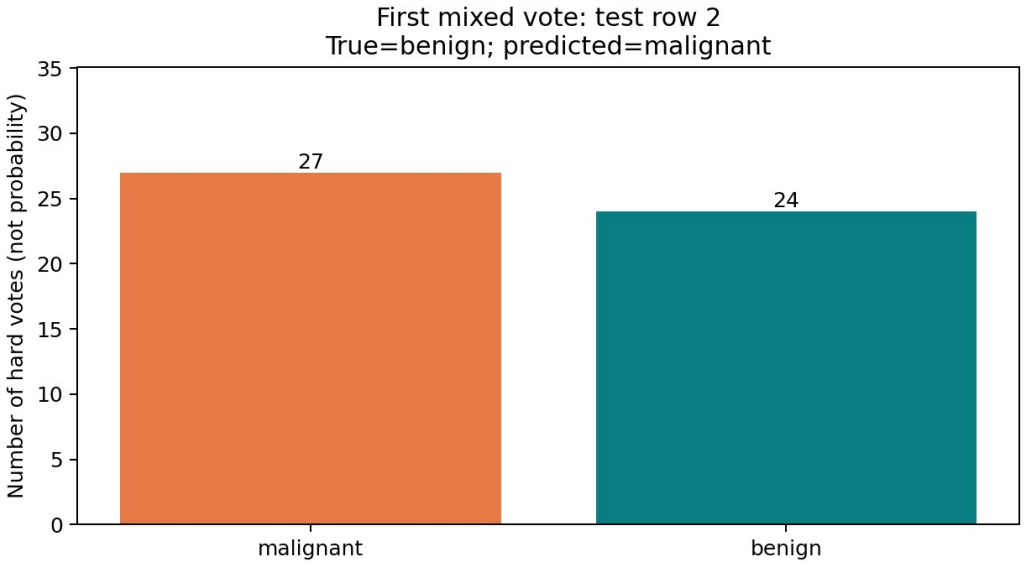
Sự cải thiện có thể được giải thích bởi việc tổng hợp các cây học trên các mẫu khác nhau làm giảm tác động của một cây sai riêng lẻ. Tuy nhiên, thực nghiệm này không đo trực tiếp tương quan lỗi hay tách riêng tác dụng của từng thành phần. Đó là cách giải thích phù hợp với cơ chế, chưa phải kết luận nhân quả đã được kiểm chứng riêng.

OOB của Bagging ở seed tham chiếu

OOB accuracy = 401/426 = 94,13%; coverage = 100,00%. Mỗi hàng nhận từ 8 đến 34 phiếu từ những cây chưa thấy nó. Số phiếu không giống nhau giữa các hàng và không phải đủ 51 phiếu. OOB được đọc cùng coverage và giới hạn liên quan đến bước chọn depth đã nêu.

4.8. Một trường hợp tổ hợp vẫn sai

Để nhìn rõ cơ chế thay vì chọn ví dụ đẹp, chương trình lấy mẫu test đầu tiên có phiếu ở cả hai lớp. Mẫu tại vị trí test 2 ứng với hàng 291 trong dữ liệu gốc, đều đếm từ 0; đây là chỉ số hàng, không phải mã bệnh nhân. Nhãn thật là lành tính.



Hình 3. Ví dụ bất đồng thật: 27 phiếu ác, 24 phiếu lành; tổ hợp dự đoán sai.

Có 27 cây chọn ác tính và 24 cây chọn lành tính, nên hard voting chọn ác tính. Do nhãn thật là lành tính, đây là dương tính giả (FP). Ví dụ chứng minh việc có đa số phiếu không đảm bảo dự đoán đúng. Tỷ lệ 27/51 không được diễn giải là xác suất mắc bệnh đã hiệu chỉnh.

4.9. Giới hạn và hướng cải thiện vừa sức

- Độ sâu được chọn bằng cây đơn rồi dùng chung. Có thể cải thiện bằng CV riêng cho mỗi mô hình, vẫn giữ test ngoài quá trình chọn cấu hình.
- 51 cây là cấu hình minh họa cố định, chưa chứng minh tối ưu. Khảo sát số cây bằng train/CV là hướng mở rộng hợp lý; không chọn dựa trên test.
- Mười holdout có giao nhau; SD chỉ mô tả dao động. Cần dữ liệu ngoài hoặc thiết kế đánh giá khác để kết luận rộng hơn.
- Dữ liệu hiện có chưa đại diện mọi quần thể thực tế. Kết quả học thuật và các ca bỏ sót còn tồn tại không đủ để sử dụng cho quyết định lâm sàng.

KẾT LUẬN

Báo cáo đã trình bày Ensemble Learning qua ý tưởng kết hợp những mô hình vừa đủ tốt vừa có sự đa dạng. Bagging hiện thực ý tưởng đó bằng bootstrap và tổng hợp dự đoán. Ví dụ năm quan sát nối trực tiếp dữ liệu, các tập bootstrap, ngưỡng cây, phiếu của cùng một mẫu và kết quả cuối cùng; phép tính OOB còn cho thấy vì sao phải phân biệt mẫu có và không có phiếu hợp lệ.

Mã ManualBagging tự thực hiện lấy mẫu, hard voting và OOB, trong khi sử dụng cây quyết định của scikit-learn làm thành phần cơ sở. Quy trình thực nghiệm giữ test ngoài việc chọn độ sâu, so sánh trên cùng các lần chia và xuất dữ liệu đủ để kiểm tra kết quả. Cách triển khai vì vậy thể hiện được cơ chế tổ hợp thay vì chỉ gọi sẵn một mô hình Bagging.

Trên mười holdout đã khảo sát, accuracy trung bình của Bagging là 94,76 ± 1,59%, cao hơn cây đơn trong bộ thí nghiệm này. Kết quả seed 42 cho thấy Bagging vẫn bỏ sót bốn ca ác tính và ví dụ bất đồng cho thấy tổ hợp vẫn có thể sai. Kết luận thích hợp là Bagging có lợi ích trên dữ liệu và cấu hình đã khảo sát, đồng thời cần giữ giới hạn khi suy rộng.

Ba ý chính để trình bày

- Bootstrap tạo các tập học khác nhau; đó là nguồn đa dạng của các cây trong Bagging.
- Hard voting đếm phiếu của cùng một quan sát; đa số phiếu không đồng nghĩa chắc chắn đúng hay xác suất bệnh.
- Đánh giá phải tách train/test, đọc các loại lỗi và mô tả trung thực phạm vi của kết quả.

Dữ liệu kiểm chứng đi kèm

results.json lưu cấu hình, phiên bản, điểm CV, OOB và trace; metrics.csv lưu lần tham chiếu; repeated_metrics.csv và summary_metrics.csv lưu điểm từng seed và thống kê; predictions.csv lưu nhãn thật, dự đoán và số phiếu từng mẫu test. Các ảnh trong báo cáo được sinh từ cùng lượt chạy đầy đủ. examples/toy_bagging.py tái lập phép tính của Chương 2.

TÀI LIỆU THAM KHẢO

[1] Breiman, L. (1996). Bagging Predictors. Machine Learning, 24, 123–140. https://doi.org/10.1007/BF00058655

[2] Breiman, L. (2001). Random Forests. Machine Learning, 45, 5–32. https://doi.org/10.1023/A:1010933404324

[3] scikit-learn developers. Ensemble methods. Tài liệu hướng dẫn chính thức. https://scikit-learn.org/stable/modules/ensemble.html

[4] scikit-learn developers. load_breast_cancer — Breast Cancer Wisconsin (Diagnostic). Tài liệu API và mô tả dữ liệu. https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_breast_cancer.html

[5] scikit-learn developers. Decision Trees. Tài liệu về tiêu chí chia, Gini và triển khai cây. https://scikit-learn.org/stable/modules/tree.html

[6] Freund, Y., & Schapire, R. E. (1997). A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting. Journal of Computer and System Sciences, 55(1), 119–139. https://doi.org/10.1006/jcss.1997.1504

[7] Wolpert, D. H. (1992). Stacked Generalization. Neural Networks, 5(2), 241–259. https://doi.org/10.1016/S0893-6080(05)80023-1

Nguồn của kết quả và ranh giới đóng góp

Các bảng số liệu và hình thực nghiệm do mã trong repository sinh từ dữ liệu của scikit-learn, không lấy điểm benchmark từ các bài báo trên. Nguồn [1] và [2] cung cấp cơ sở phương pháp; [3]–[5] giải thích hành vi thư viện và dữ liệu; [6]–[7] phục vụ phần tổng quan. Cây cơ sở và Random Forest thuộc scikit-learn; mã minh họa tập trung vào bootstrap, tổng hợp hard-vote và kiểm tra OOB.

Thông tin phiên bản của lần chạy: Python 3.12.14; NumPy 2.3.5; scikit-learn 1.8.0; matplotlib 3.10.8.